

# LLMs: A Driving Force in Next Generation Digital Design Automation

Matheus T. Moreira<sup>†</sup>, Aram H. Markosyan<sup>‡</sup>, Chris Cummins<sup>‡</sup>, Warren Hunt<sup>‡</sup>, Gabriel Synnaeve<sup>‡</sup>, Edith Beigne<sup>†</sup>

<sup>†</sup>Reality Labs, <sup>‡</sup>Fundamental AI Research

Meta Platforms, Inc. Menlo Park, CA, USA

matheustrev@meta.com

**Abstract**—Innovations in generative artificial intelligence (GenAI), particularly large language models (LLMs), are poised to revolutionize silicon design automation. This paper explores the transformative potential of LLMs in automating and enhancing various tasks within the silicon design process. It reviews the current applications of LLMs and their potential to automate silicon design tasks, proposing applications, providing a qualitative analysis of the readiness of the technology to support these applications and setting directions for future research.

**Index Terms**—Large language models, design automation.

## I. INTRODUCTION

The growing capabilities of artificial intelligence (AI) models are impacting fields like software development and could revolutionize digital silicon design. LLMs are particularly promising for silicon design due to their ability to process and generate text, including code and technical documents. This is beneficial, as engineers often need to handle text-based data. Advancements in LLM agents offer promising opportunities for integrating LLMs into assisted or autonomous systems, enhancing task planning and tool execution [1]. In silicon design, EDA tools can be integrated into these systems to validate outputs from LLMs, ensuring compliance with various requirements and improving design accuracy and reliability.

In related works, LLMs were applied to automate tasks such as circuit verification and debugging [3], [4], testbench generation [5], formal verification [6], [7], circuit topology synthesis [8], EDA tool integration [9]–[12], layout generation [13], high-level synthesis [14], documentation [15], [16] and RTL code generation [17]–[21], among other tasks. Recent studies have explored LLM agents with simulators to iteratively generate and verify RTL code, enhancing the ability of these systems to produce functionally correct RTL [22]–[26], [28], [29]. A notable limitation highlighted by most of these works is the lack of high-quality silicon datasets, which hinders the development and evaluation of LLMs for silicon design tasks. Access to these datasets is crucial for validating hypotheses, benchmarking performance, and refining prompt engineering strategies. This scarcity is largely due to the proprietary nature of silicon data, unlike the open-source availability in software development. However, there is a growing trend towards creating publicly available silicon datasets and benchmarks [17]–[21], [30]–[35], which can mitigate data scarcity and promote LLM-driven silicon design.

In this paper we contend that LLMs are a driving force in next generation digital design automation. We identify a

set of basic capabilities for LLMs to support automating digital design and explore how they can benefit from available technology, describing associated risks and identifying gaps. Table I provides an overview of these capabilities, identifying main challenges and feasibility of implementing them with current technological capabilities. After discussing these basic capabilities, we discuss how they can be used to develop more complex and capable digital design automation using agentic workflows and LLM agents. Finally, we identify gaps and set directions for required research and development in this field.

## II. BACKGROUND

LLMs are a class of AI models designed to understand and generate human-like text [2]. These models are typically based on transformer architectures, which enable them to process and generate text considering the context of words in a sentence. LLMs are trained on vast amounts of text data, allowing them to learn patterns, grammar, and even reasoning and planning [36]. The training process involves adjusting model parameters to minimize the difference between its predictions and the actual text in the training data. Evaluation of these models is typically conducted using benchmarks that measure their performance on tasks such as text completion, translation, and question answering. Interfacing with them is primarily done through prompting, where the input is text data to guide the model's response. The prompt can be a question, a statement, or a partial sentence, and the LLM generates text data as output that is a continuation or a relevant response based on the input. The generated output is probabilistic, meaning that the same prompt can yield different responses depending on internal state and the randomness introduced during generation.

Pre-trained LLMs [37], such as the recently announced LLaMa 4 models [38], have been fine-tuned for various applications and are currently available. These models are capable of generalizing to many tasks due to their extensive training on diverse datasets, which enables them to capture a broad spectrum of linguistic patterns and contextual nuances. This generalization is further enhanced by their ability to leverage transfer learning, where knowledge gained from one task is applied to new, related tasks. As a result, pre-trained LLMs can efficiently adapt to specific domains with no or minimal additional training, offering significant opportunities for innovation in fields like silicon design. Prompt engineering is the practice of designing and refining prompts to elicit desired responses from LLMs and can significantly enhance

TABLE I  
BASIC CAPABILITIES FOR LLMs TO SUPPORT AUTOMATING DIGITAL DESIGN AND EXAMPLE APPLICATIONS. MAIN CHALLENGES ARE RISKS OF ADOPTING EXISTING TECHNOLOGY AND FEASIBILITY IDENTIFIES THE PRACTICALITY OF A CAPABILITY WITH EXISTING TECHNOLOGY.

| Capability             | Example Applications                                                                                                                                                                                                                 | Main Challenges                                                                                                                                                                                            | Feasibility                       |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Documentation          | <ul style="list-style-type: none"> <li>- Read query, search documents and answer</li> <li>- Write document from instructions + context</li> <li>- Report missing requirements on a document</li> </ul>                               | <ul style="list-style-type: none"> <li>- Limited availability of models, datasets and benchmarks</li> </ul>                                                                                                | High [51]–[53]                    |
| Reasoning and Planning | <ul style="list-style-type: none"> <li>- Create plan to code RTL of a design from a specification</li> <li>- Create a verification plan from a set of requirements</li> <li>- Create plan to optimize a low power design</li> </ul>  | <ul style="list-style-type: none"> <li>- Adaptability to technological advancements</li> <li>- Validation of model responses</li> <li>- Limited availability of models, datasets and benchmarks</li> </ul> | Low [36]                          |
| Coding                 | <ul style="list-style-type: none"> <li>- Read query, search codebase and answer</li> <li>- Write RTL code from instructions + context</li> <li>- Translate synthesis scripts between EDA tools</li> </ul>                            | <ul style="list-style-type: none"> <li>- Token limitation</li> <li>- Validation of model responses</li> <li>- Limited availability of models, datasets and benchmarks</li> </ul>                           | Medium [19], [25] [44]–[46], [48] |
| EDA Tool               | <ul style="list-style-type: none"> <li>- Setup simulation tool and run from instructions + context</li> <li>- Suggest bug fixes from behavioral simulation log</li> <li>- Suggest design optimizations from timing report</li> </ul> | <ul style="list-style-type: none"> <li>- Non-standard interfaces</li> <li>- Limited availability of models, datasets and benchmarks</li> </ul>                                                             | Medium [10], [49]                 |

the quality and relevance of the generated output, avoid the need for additional training. Techniques in prompt engineering include using specific keywords, providing context, examples of expected outcomes, and structuring prompts in a way that aligns with the training data used in the model. Retrieval systems complement LLMs by providing access to external information that the model might not have been exposed to during training. Retrieval-Augmented Generation (RAG) is a technique that combines LLMs with retrieval systems to enhance their performance [41]. In a RAG system, the model first retrieves relevant documents or data based on the input prompt and then uses this information to generate a more informed and accurate response.

An LLM agentic workflow is a software system that uses LLMs and tools to complete tasks statically through pre-defined processes [42]. LLM agents also have access to LLMs and tools that they can use to complete tasks, but are able to dynamically complete these tasks through flexible autonomous decision-making, supported by LLMs [42]. These agents can be categorized into several architectures [43]. For example, reactive agents respond to environmental stimuli without internal state or planning, making them suitable for real-time control systems, such as generating designs based on predefined rules and constraints in digital silicon design. Self-reflection agents, on the other hand, have the ability to reason about their own internal states and decision-making processes, allowing them to adapt to changing circumstances and improve performance over time. Also, learning-based architectures enable agents to learn from experience and improve performance and multi-agent systems involve multiple agents interacting and coordinating to achieve common goals. These architectures leverage LLM capabilities to reason and create plans that outline tasks to achieve a goal [36], [39], [40] by executing those tasks.

### III. BASIC CAPABILITIES

In digital design, relevant information like requirements, specifications and plans is communicated through natural language in well structured documents. The multiple teams of humans involved in the process of going from concept to tapeout are responsible for writing these documents. These teams work in an organized process that executes sequences of pre-defined tasks to transform data that constitute a design

flow. To perform their work, they consume documentation from dependencies like partner teams or third-party providers to create architectures and plans, producing more documentation. This requires iterations of document reviews, as teams converge in their architectures and plans. These produced architectures are typically used to build circuits using different code languages. Plans describe how to perform certain tasks, like verifying a design or synthesizing it, and the target quality metrics, like coverage and maximum area.

Coding is how engineers express design intent, i.e. define circuit requirements, describe logic behavior and integrate tools in implementation flows. It is how they build circuits that support specified architectures and drive tools to perform the required data transformations through the steps of a design flow. In order to code efficiently, engineers typically must learn about code dependencies and revise associated documentation. This is because silicon design is complex and expensive, so it is common for teams to rely on hierarchical design approaches. Code is one of the key drivers of EDA tools, along with libraries of dependencies, such as cell libraries and process design kits. These tools are the means used by engineers to perform a large portion of their technical tasks and transform data, like simulating behavior, synthesizing circuits and analyzing test coverage, to name a few. The output of these tools are either in the form of code or logs and reports. While code produced by EDA tools is typically not immediately interesting to designers, it is the way that different tools communicate in the different steps of the design flow. Logs and reports, on the other hand are important resources for engineers to debug their code and analyze the quality of their designs. For example, a simulation log provides insights on how to fix the RTL code or the verification code, while a timing report provides insights on the expected performance of the design after it is fabricated. Even though, engineers do rely on interactive use of EDA tools to debug and optimize their designs, a large portion of the work is done by analyzing these logs and reports. In fact, sign-off processes typically rely on articulating design readiness through logs and reports.

This high level description of a digital design flow is abstract, but enables us to identify the basic capabilities that can be developed using LLMs to automate different tasks at different stages of a design flow. Table I shows the identified

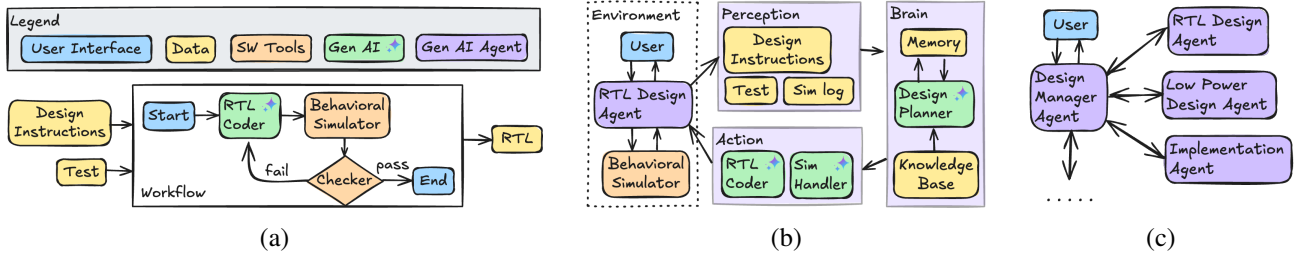


Fig. 1. Integration of basic capabilities to automate digital design tasks. In (a), we show an agentic workflow to generate RTL code from a set of instructions to design the circuit using natural language, along with a test to simulate generated code. In (b) we show an LLM agent capable of planning and designing RTL and in (c) we show a generic multi-agent system to automate digital design tasks.

list and presents example applications. It also describes the main challenges of automating them and provides a score for the feasibility of automating a capability using LLMs. High feasibility means they can be automated using existing technology. Medium feasibility means a capability can be partially automated, but requires further development or performance optimization. Low feasibility means a requires significant advancements in LLMs to be automated.

For documentation applications, RAG systems can read queries in natural language, find relevant information in associated input documents and produce accurate and contextually relevant responses in natural language. Although available models were not trained on extensive silicon-specific data, previous works show the ability of LLMs to generalize through effective prompt engineering [51]–[53]. RAG systems are also well-suited to help automating coding tasks due to the ability of retrieval systems to search for relevant context and capability of LLMs to understand and generate code [19], [44]–[46]. While prompt engineering can help available models to generalize, silicon code can be large and exceed tokenization limits, restricting context provision and output size. To address this, large code can be divided into smaller modules or processed hierarchically, enabling LLMs to focus on specific sections. Another challenge is ensuring generated code meets requirements like correct syntax or functionality. Using EDA tools to validate code effectively is a mitigation strategy that has demonstrated good results [25].

Existing works show the ability of LLMs to drive tools [49] and analyze logs [54]. Unfortunately, EDA tools typically have their own unique set of commands, configurations, and outputs, making it difficult to standardize interfaces. RAG systems can help to some extent by retrieving relevant context from documents like a tool manual or from code, and providing insightful information in natural language. In fact, recent works, such as [10], [54], show promising results for this capability. LLMs have also demonstrated their ability to tackle reasoning and planning problems [36]. Yet, applying them to plan for digital design faces challenges, as the lack of domain-specific data in the training datasets of existing LLMs makes it harder for them to generalize and validating plans generated by these models is not a trivial task. Furthermore, technology nodes, libraries and EDA tools evolve at a steady pace, and require reassessment of design choices and flows roughly in an yearly basis, so LLMs must adapt to these changes.

#### IV. AGENTIC WORKFLOWS AND LLM AGENTS

Some of these basic capabilities are already being explored in research works and adopted as specific applications in design flows. They represent the first wave of LLM automation and can free engineers from monotonous and repetitive tasks to focus on more creative and challenging tasks. In this first wave, tasks can be automated using agentic workflows, as the example in Figure 1 (a). The diagram shows a workflow where RTL generation can be achieved by providing a set of instructions to design the circuit using natural language, along with a test to simulate generated code. It starts with an LLM that creates RTL code from input instructions and simulates it, using an input test. If simulation passes, the workflow outputs the RTL code to the user. Otherwise, it iteratively uses the LLM to fix the RTL code and simulate until simulation passes or the workflow reaches a maximum iteration number.

The integration of LLMs into digital design flows in the form of agentic workflows presents a transformative opportunity to automate various tasks, enhancing efficiency and innovation. Yet, we believe that it is only the beginning of a more transformative wave. As LLM get more capable on digital design tasks, particularly on reasoning and planning, they will be able to autonomously drive complex tasks. As an example, Figure 1 (b) shows the general structure for an LLM agent [55] capable of interacting with a user to generate RTL code. Note that this is different than the agentic workflow example, because here the agent is responsible of creating a plan that can include retrieving relevant information from a knowledge base, using LLMs to generate code and execute tools, storing and recalling information from a memory and interacting with the user to validate assumptions or ask clarifying questions. This plan is what the agent uses to achieve the goal of creating RTL, versus the statically defined workflow of Figure 1 (a).

Figure 1 (c) shows a basic structure for an even more advanced stage, where multiple of these LLM agents can work together to perform more complex tasks. Assuming this example, a design manager agent could interact with a user to collect functional, power, area, and performance requirements of a low power design. It could then interact with an RTL design agent to generate RTL code and with an implementation agent that could write synthesis recipes and take the design through synthesis, iteratively optimizing the process by analyzing reports. At the end, this multi-agent system would provide the generated code along with

documentation and a detailed report to the designer, explaining the design choices and reporting the obtained power, area, and performance metrics. Note that even though we are showing a vertical architecture for the multi-agent example, this is just for demonstration purposes and systems can be built with horizontal architectures as well [43]. Other examples of multi-agent systems include systems that automate the entire verification process or optimize design for low power.

## V. DISCUSSION AND CONCLUSIONS

The presented basic capabilities represent the first wave of LLM automation in digital design and some of them are already being explored and adopted for very specific applications in design flows, mostly as agentic workflows. A global challenge for these capabilities is the limited availability of models, datasets and benchmarks tailored for silicon applications. The proprietary nature of silicon design data limits the availability of high-quality datasets, hindering the development of LLMs capable of effectively supporting related tasks. To mitigate these challenges, the development of publicly available silicon datasets and benchmarks is crucial, as they provide the necessary resources for training and evaluating LLMs for specific goals, further improving performance.

Even though LLMs provide exciting opportunities for design automation, adopting them in design flows can be challenging due to the complexities of AI technologies and the silicon design process. Key issues include runtime, cost and performance, where LLMs must scale to handle complex designs without excessive runtime or associated costs, as these are relevant metrics in design automation tools and flows. For that, novel and smaller models tailored for specific applications are required. Also, it is important to mention that the statistical nature of AI introduces challenges in repeatability and predictability, limiting their scope to tasks that tolerate such a nature. This can be a challenge for using LLMs to develop EDA tools, but is a perfect fit to automate human-driven tasks, which also face repeatability and predictability challenges. Another important challenge is the lack of standardized interfaces for EDA tools, and there is a need to enable agents to drive tool execution from design intents more abstract than code.

More recently, breakthroughs in code-focused LLMs have been propelled by online reinforcement learning (RL) loops and execution feedback mechanisms that couple model training with real-time correctness and performance signals. Rather than learning solely from static code corpora, modern code LLMs increasingly train with feedback from code execution, e.g. running generated code against test cases and using the outcomes as a reward signal to update the model. This paradigm extends into inference, where models iteratively refine outputs by receiving feedback (compiler errors, unit test results) to adjust solutions. This feedback-driven learning has rapidly become a critical driver of improved performance on programming tasks [56]. Empirical results show that models trained to leverage execution feedback can achieve higher accuracy with fewer attempts, indicating a substantial efficiency boost in reliability and problem-solving [56].

These ideas are exemplified in several recent systems. DeepSeek-R1 (and its vision-language counterpart DeepSeek-VL R1) is built on large-scale post-training via RL, which encourages the model to develop robust reasoning patterns – notably exhibiting self-verification and reflective debugging abilities in solving code and math problems [57]. DeepCoder-14B from Together AI likewise employs an online RL pipeline: it fine-tunes a 14B parameter coder with distributed reward optimization, executing the model’s code in sandbox environments and reinforcing correct solutions. This approach yielded an absolute 8% accuracy gain on a competitive coding benchmark compared to the supervised baseline [58]. Similarly, Meta’s RLEF framework trains models to use test outcomes as hints for incremental improvement, effectively turning code generation into an iterative decision process [56]. Across these systems, the common thread is a shift toward feedback-centric adaptation – models that learn from execution traces and rewards tend to demonstrate superior reasoning, reliability, and alignment with the desired solution, underlining the importance of interactive learning in next-generation digital design automation.

Recent benchmarks such as SWEbench and SWE-agent emphasize end-to-end coding challenges that mirror real software development cycles. These tasks present an LLM with an entire codebase and a natural-language issue description, requiring the model to comprehend existing code, implement a fix, and verify it via tests [59], [60]. By capturing this complex, multi-stage problem-solving process, they provide a rigorous gauge of real-world applicability. Notably, results on these benchmarks have improved rapidly: state-of-the-art agents now resolve roughly one-fifth of the full SWEbench problems (and 43% on a simplified “Lite” subset) [61], and after filtering out ambiguous cases the best systems exceed 50% success on the human-validated SWEbench. Crucially, these gains have been driven by RL techniques both during training and at inference, which inject iterative feedback into the coding process. For example, augmenting GPT-4 with an advanced code-editing scaffold (“CodeR”) boosted its solve rate on SWEbench-Lite from 2.7% to 28.3% [61], underscoring the impact of inference-time feedback loops on performance. Yet, to enable these recent developments in silicon we will need to overcome challenges related to the proprietary nature of data.

In conclusion, LLMs are set to be a driving force in next-generation digital design automation. Their ability to process and generate human-like text, combined with retrieval systems and agent architectures, offers significant opportunities to automate and enhance silicon design tasks, but there are many challenges to be solved. Overcoming these challenges necessitates a multidisciplinary approach, combining AI, silicon design, and software engineering expertise, with research and collaboration between academia and industry. As the AI field continues to evolve rapidly, new models may bring additional capabilities, enabling the resolution of some of these challenges. This progress will allow engineers to focus on more abstract and innovative problems, ultimately pushing silicon technology forward.

## REFERENCES

- [1] Y. Sheng, S. Gandhe, B. Kanagal, N. Edmonds, Z. Fisher, S. Tata, and A. Selvan, Measuring an LLM's Proficiency at using APIs: A Query Generation Strategy, 2024, Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 5680-5689, 2024, doi: 10.1145/3637528.3671592.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention Is All You Need, 2023, arXiv:1706.03762, <https://arxiv.org/abs/1706.03762>.
- [3] Y. Hu, J. Ye, K. Xu, J. Sun, S. Zhang, X. Jiao, D. Pan, J. Zhou, N. Wang, W. Shan, X. Fang, X. Wang, N. Guan, and Z. Jiang, UVLLM: An Automated Universal RTL Verification Framework using LLMs, 2024, arXiv:2411.16238, <https://arxiv.org/abs/2411.16238>.
- [4] K. Qayyum, M. Hassan, S. Ahmadi-Pour, C. K. Jha, and R. Drechsler, From Bugs to Fixes: HDL Bug Identification and Patching using LLMs and RAG, 2024 IEEE LLM Aided Design Workshop (LAD), 2024, pp. 1-5, doi: 10.1109/LAD62341.2024.10691874.
- [5] R. Qiu, G. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design, 2024, arXiv:2411.08510, <https://arxiv.org/abs/2411.08510>.
- [6] M. Kang, M. Liu, G. Hamad, S. Suhaib, and H. Ren, FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware, 2024, arXiv:2410.23299, <https://arxiv.org/abs/2410.23299>.
- [7] M. Orenes-Vera, M. Martonosi, and D. Wentzlaff, Using LLMs to Facilitate Formal Verification of RTL, 2023, arXiv:2309.09437, <https://arxiv.org/abs/2309.09437>.
- [8] P. Vijayaraghavan, L. Shi, E. Degan, and X. Zhang, CIRCUITSYNTH: Leveraging Large Language Models for Circuit Topology Synthesis, 2024, arXiv:2407.10977, <https://arxiv.org/abs/2407.10977>.
- [9] B. Wu, U. Sharma, S. Kankipati, A. Yadav, B. George, S. Guntupalli, A. Rovinski, and V. Chhabria, EDA Corpus: A Large Language Model Dataset for Enhanced Interaction with OpenROAD, 2024, arXiv:2405.06676, <https://arxiv.org/abs/2405.06676>.
- [10] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, ChatEDA: A Large Language Model Powered Autonomous Agent for EDA, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 43, no. 10, pp. 3184-3197, Oct. 2024, doi: 10.1109/TCAD.2024.3383347.
- [11] L. Shi, M. Kazda, B. Sears, N. Shropshire, and R. Puri, Ask-EDA: A Design Assistant Empowered by LLM, Hybrid RAG and Abbreviation Dehallucination, 2024, arXiv:2406.06575, <https://arxiv.org/abs/2406.06575>.
- [12] S. Qiu, B. Tan, and H. Pearce, LLM-aided explanations of EDA synthesis errors, 2024 IEEE LLM Aided Design Workshop (LAD), 2024, pp. 1-6, doi: 10.1109/LAD62341.2024.10691721.
- [13] C. Ho, and H. Ren, Large Language Model (LLM) for Standard Cell Layout Design Optimization, 2024, arXiv:2406.06549, <https://arxiv.org/abs/2406.06549>.
- [14] L. Collini, S. Garg, and R. Karri, C2HLSC: Can LLMs Bridge the Software-to-Hardware Design Gap?, 2024, arXiv:2406.09233, <https://arxiv.org/abs/2406.09233>.
- [15] S. Fernando, R. Kunzelmann, D. Lopera, J. Al Halabi, and W. Ecker, Boosting Productivity of Hardware Documentation Using Large Language Models, 2024 IEEE LLM Aided Design Workshop (LAD), 2024, pp. 1-1, doi: 10.1109/LAD62341.2024.10691698.
- [16] M. Li, W. Fang, Q. Zhang, and Z. Xie, SpecLLM: Exploring Generation and Review of VLSI Design Specification with Large Language Model, 2024, arXiv:2401.13266, <https://arxiv.org/abs/2401.13266>.
- [17] A. Allam and M. Shalan, RTL-Repo: A Benchmark for Evaluating LLMs on Large-Scale RTL Design Projects, 2024, arXiv:2405.17378, <https://arxiv.org/abs/2405.17378>.
- [18] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model, 2023, arXiv:2308.05345, <https://arxiv.org/abs/2308.05345>.
- [19] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, Revisiting VerilogEval: A Year of Improvements in Large-Language Models for Hardware Code Generation, 2025, arXiv:2408.11053, <https://arxiv.org/abs/2408.11053>.
- [20] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, Benchmarking Large Language Models for Automated Verilog RTL Code Generation, 2022, arXiv:2212.11140, <https://arxiv.org/abs/2212.11140>.
- [21] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. Lin, MG-Verilog: Multi-grained Dataset Towards Enhanced LLM-assisted Verilog Generation, 2024, arXiv:2407.01910, <https://arxiv.org/abs/2407.01910>.
- [22] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, MAGE: A Multi-Agent Engine for Automated RTL Code Generation, 2024, arXiv:2412.07822, <https://arxiv.org/abs/2412.07822>.
- [23] A. Nakkab, S. Zhang, R. Karri, and S. Garg, Rome was Not Built in a Single Step: Hierarchical Prompting for LLM-based Chip Design, 2024, arXiv:2407.18276, <https://arxiv.org/abs/2407.18276>.
- [24] M. Islam, H. Sami, P. Gaillardon, and V. Tenace, EDA-Aware RTL Generation with Large Language Models, 2024, arXiv:2412.04485, <https://arxiv.org/abs/2412.04485>.
- [25] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, AutoChip: Automating HDL Generation Using LLM Feedback, 2024, arXiv:2311.04887, <https://arxiv.org/abs/2311.04887>.
- [26] H. Huang, Z. Lin, Z. Wang, X. Chen, K. Ding, and J. Zhao, Towards LLM-Powered Verilog RTL Assistant: Self-Verification and Self-Correction, 2024, arXiv:2406.00115, <https://arxiv.org/abs/2406.00115>.
- [27] N. Wang, B. Yao, J. Zhou, X. Wang, Z. Jiang, and N. Guan, Large Language Model for Verilog Generation with Golden Code Feedback, 2024, arXiv:2407.18271, <https://arxiv.org/abs/2407.18271>.
- [28] B. Wang, Q. Xiong, Z. Xiang, L. Wang, and R. Chen, RTLsquad: Multi-Agent Based Interpretable RTL Design, 2025, arXiv:2501.05470, <https://arxiv.org/abs/2501.05470>.
- [29] B. Nadimi and H. Zheng, A Multi-Expert Large Language Model Architecture for Verilog Code Generation, 2024 IEEE LLM Aided Design Workshop (LAD), 2024, pp. 1-5, doi: 10.1109/LAD62341.2024.10691683.
- [30] N. Xu, Z. Zhang, L. Qi, W. Wang, C. Zhang, Z. Ren, H. Zhang, X. Cheng, Y. Zhang, Z. Liu, Q. Wei, S. Wu, L. Yang, Q. Lu, Y. Ma, M. Zhao, J. Liu, Y. Song, X. Geng, and J. Yang, ChipExpert: The Open-Source Integrated-Circuit-Design-Specific Large Language Model, 2024, arXiv:2408.00804, <https://arxiv.org/abs/2408.00804>.
- [31] J. Bhandari, V. Bhat, Y. He, H. Rahmani, S. Garg, and R. Karri, Masala-CHAI: A Large-Scale SPICE Netlist Dataset for Analog Circuits by Harnessing AI, 2025, arXiv:2411.14299, <https://arxiv.org/abs/2411.14299>.
- [32] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, D. Song, D. Lin, X. Zhang, and Y. Liang, OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection, 2024, arXiv:2407.16237, <https://arxiv.org/abs/2407.16237>.
- [33] M. DeLorenzo, V. Gohil, and J. Rajendran, CreativEval: Evaluating Creativity of LLM-Based Hardware Code Generation, 2024, arXiv:2404.08806, <https://arxiv.org/abs/2404.08806>.
- [34] L. Skelic, Y. Xu, M. Cox, W. Lu, T. Yu, and R. Han, CIRCUIT: A Benchmark for Circuit Interpretation and Reasoning Capabilities of LLMs, 2025, arXiv:2502.07980, <https://arxiv.org/abs/2502.07980>.
- [35] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation, 2025, arXiv:2503.15112, <https://arxiv.org/abs/2503.15112>.
- [36] L. Kang, Z. Zhao, D. Hsu, and W. S. Lee, On the Empirical Complexity of Reasoning and Planning in LLMs, 2024, arXiv:2404.11041, <https://arxiv.org/abs/2404.11041>.
- [37] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, LLaMA: Open and Efficient Foundation Language Models, 2023, arXiv:2302.13971, <https://arxiv.org/abs/2302.13971>.
- [38] Llama, Accessed 2025-04-09, <https://www.llama.com>.
- [39] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, Understanding the planning of LLM agents: A survey, 2024, arXiv:2402.02716, <https://arxiv.org/abs/2402.02716>.
- [40] T. Birr, C. Pohl, A. Younes, and T. Asfour, AutoGPT+P: Affordance-based Task Planning using Large Language Models, Robotics: Science and Systems Foundation, 2024, doi: 10.15607/rss.2024.xx.112.
- [41] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, 2021, arXiv:2005.11401, <https://arxiv.org/abs/2005.11401>.
- [42] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, AFlow: Automating Agentic Workflow Generation, 2025, arXiv:2410.10762, <https://arxiv.org/abs/2410.10762>.
- [43] T. Masterman, S. Besen, M. Sawtell, and A. Chao, The Landscape of Emerging AI Agent Architectures for Reasoning,

- Planning, and Tool Calling: A Survey, 2024, arXiv:2404.11584, <https://arxiv.org/abs/2404.11584>.
- [44] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Hebgens Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, Evaluating Large Language Models Trained on Code, arXiv:2107.03374, <https://arxiv.org/abs/2107.03374>.
- [45] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, A systematic evaluation of large language models of code, Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, 2022, pp. 1-10, doi: 10.1145/3520312.3534862.
- [46] R. Ma, Y. Yang, Z. Liu, J. Zhang, M. Li, J. Huang, and G. Luo, Guojie, VerilogReader: LLM-Aided Hardware Test Generation, 2024 IEEE LLM Aided Design Workshop (LAD), 2024, pp. 1-5, doi: 10.1109/LAD62341.2024.10691801.
- [47] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, A Survey on Large Language Models for Code Generation, 2024, arXiv:2406.00515, <https://arxiv.org/abs/2406.00515>.
- [48] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, Towards Translating Real-World Code with LLMs: A Study of Translating to Rust, 2024, arXiv:2405.11514, <https://arxiv.org/abs/2405.11514>.
- [49] C. Tao, X. Fan, and Y. Yang, Harnessing LLMs for API Interactions: A Framework for Classification and Synthetic Data Generation, 2024, arXiv:2409.11703, <https://arxiv.org/abs/2409.11703>.
- [50] A. Zhong, M. Dengyao, G. Liu, J. Liu, Q. Lu, Q. Zhou, J. Wu, Q. Li, and Q. Wen, LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models, 2024, arXiv:2408.13727, <https://arxiv.org/abs/2408.13727>.
- [51] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, and J. Wen, Large Language Models for Information Retrieval: A Survey, 2024, arXiv:2308.07107, <https://arxiv.org/abs/2308.07107>.
- [52] K. Tyser, B. Segev, G. Longhitano, X. Zhang, Z. Meeks, J. Lee, U. Garg, N. Belsten, A. Shporer, M. Udell, D. Te'eni, and I. Drori, AI-Driven Review Systems: Evaluating LLMs in Scalable and Bias-Aware Academic Reviews, 2024, arXiv:2408.10365, <https://arxiv.org/abs/2408.10365>.
- [53] S. Chakrabarty, and S. Pal, Free and Customizable Code Documentation with LLMs: A Fine-Tuning Approach, 2024, arXiv:2412.00726, <https://arxiv.org/abs/2412.00726>.
- [54] A. Gupta, Leveraging Large Language Models for Log Mining: A New Paradigm in Data Analysis, Computer Science and Engineering, Vol. 14 No. 6, 2024, pp. 155-161. doi: 10.5923/j.computer.20241406.05.
- [55] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, The Rise and Potential of Large Language Model Based Agents: A Survey, 2023, arXiv:2309.07864, <https://arxiv.org/abs/2309.07864>.
- [56] J. Gehring, K. Zheng, J. Copet, V. Mella, Q. Carbonneaux, T. Cohen, and G. Synnaeve, Rlef: Grounding code llms in execution feedback with reinforcement learning: arXiv preprint arXiv:2410.02089, 2024.
- [57] DeepSeek-AI, DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, arXiv preprint arXiv:2501.12948, 2025.
- [58] M. Luo, S. Tan, R. Huang, X. Shi, R. Xin, C. Cai, A. Patel, A. Ariyak, Q. Wu, C. Zhang, L. Erran Li, R. Ada Popa, I. Stoica, DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level, 2025, <https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51>.
- [59] C. E Jimenez, J. Yang, A. Wettig, A. Yao, K. Pei, O. Press, K. R Narasimhan, SWE-bench: Can Language Models Resolve Real-world Github Issues?: The Twelfth International Conference on Learning Representations, 2024, <https://openreview.net/forum?id=VTF8yNQm66>.
- [60] J. Yang, C. E Jimenez, A. Wettig, K. Lieret, A. Yao, K. R Narasimhan, O. Press, SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering: The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024, <https://arxiv.org/abs/2405.15793>.
- [61] OpenAI, 2024, <https://openai.com/index/introducing-swe-bench-verified/>.